

University of Tehran
Faculty of Electrical and Computer Engineering
Natural Language Processing - Homework 6
Coding Agent Implementation & Evaluation

Student Name: [YOUR NAME HERE]
Student ID: [YOUR ID HERE]

Instructor: Dr. Heshaam Faili
TAs: Milad Mohammadi, Amirhossein Safdarian

January 5, 2026

Contents

1 Introduction

In this assignment, we designed and implemented a **Coding Agent** capable of interacting with a codebase, debugging errors, and adding new features. Unlike traditional chatbots, this agent possesses *Agency*—the ability to interact with the file system and execute shell commands.

The agent was built using the **LangGraph** and **LangChain** frameworks, utilizing a Large Language Model (LLM) as the reasoning engine. Key features include a CLI interface, memory management, and a Human-in-the-Loop (HITL) mechanism for safety.

2 Coding Agent Implementation (100 Points)

2.1 Tool Development (25 Points)

To enable the agent to interact with the environment, I implemented a set of tools using the `@tool` decorator from LangChain. All tools verify paths to ensure they stay within the `PROJECT_ROOT` for security.

- **Read File:** Reads content of text files.
- **Create/Overwrite File:** Allows writing code to disk.
- **List Files:** vital for the agent to understand directory structure.
- **Search Files:** Uses glob patterns to find files.
- **Execute Shell:** Runs command-line operations (e.g., `pytest`).

Implementation Snippet (Tools):

```
1 # INSERT YOUR CODE FROM src/coding_agent/agent/tools.py HERE
2 # Specifically the part defining the tools
```

2.2 Graph Design and State Management (25 Points)

The core logic is implemented using **LangGraph**. I utilized a **ReAct** (Reasoning + Acting) architecture. The graph manages the state, which includes the conversation history ('messages') and the agent's scratchpad.

- **Nodes:** The graph consists primarily of the Agent node (LLM) and the Tools node.
- **Edges:** Conditional edges determine whether to loop back (call another tool) or end the turn (respond to user).
- **State:** Managed using 'MemorySaver' to persist context across turns.

Graph Implementation:

```
1 # INSERT YOUR CODE FROM src/coding_agent/agent/backend.py HERE
2 # Specifically the build_agent_graph function
```

2.3 Human In The Loop (HITL) (15 Points)

To prevent the agent from performing destructive actions (like deleting files or running dangerous commands) without permission, I implemented an interrupt mechanism.

1. The graph is configured with `interrupt_before=["tools"]`.
2. Before executing a tool, the `InteractiveSession` checks if the tool is in the `SENSITIVE_TOOLS` set (e.g., `'execute_shell'`).
3. If sensitive, the CLI prompts the user for confirmation (y/n).
4. If denied, a mock error message is injected into the state so the agent knows the action failed.

HITL Logic Snippet:

```
1 # INSERT THE _handle_tool_approval METHOD FROM session.py HERE
```

2.4 Memory Management (10 Points)

The agent maintains short-term memory of the conversation. I used a unique `thread_id` for each session to isolate contexts. The `MemorySaver` checkpointer stores the state, allowing the agent to reference previous error messages or file contents during the debugging process.

2.5 CLI Development (15 Points)

A professional Command Line Interface was developed using the **Rich** library. It features:

- Colored output for better readability.
- Panels to separate User inputs from Agent responses.
- Interactive prompts for HITL confirmations.

3 Bonus Implementation (15 Points)

3.1 Usage Tracker (5 Points)

I implemented a `TokenUsageTracker` callback that monitors input/output tokens and estimates cost based on model pricing.

Result: The total token usage and cost are displayed at the bottom of every agent response.

3.2 Smart Context Management (5 Points)

To prevent the context window from overflowing with massive file contents, a pruning mechanism was added. After a turn completes, if a `ToolMessage` (file content) exceeds 2000 characters, it is summarized/truncated in the history, keeping only the head and tail.

3.3 Session Save and Restore (5 Points)

I added `/save` and `/load` commands to the CLI. These use Python's `pickle` module to serialize the `MemorySaver` storage, allowing the user to pause debugging and resume later exactly where they left off.

4 Agent Evaluation Results

The agent was tested on three defective projects provided in the assignment. Below are the execution details and results.

4.1 Project 1: Legacy CSV Processor

Problem: The project failed to process transaction data correctly due to data type issues and lacked Pandas support.

Agent's Approach:

1. Explored `src/file_handler.py` and identified manual parsing logic.
2. Ran `pytest` and found failures in total revenue calculation.
3. Fixed the float conversion logic for the "amount" column.
4. Added handling for empty files to pass `test_handle_empty_file`.

Figure 1: Screenshot of Agent passing CSV Processor tests

Status: **PASSED** (All tests passing).

Detailed Results and Evidence:

- Commands executed:

```
1 cd test_projects/legacy_csv_processor
2 pytest -q
3
```

- Exact test output captured:

```
1 .... [100%]
2 4 passed in 0.08s
3
```

- Representative session log (stored in):

`test_projects/legacy_csv_processor/.coding_agent_logs/session_<id>.jsonl`

- Key code fix (excerpt from `src/analyzer.py`):

```

1 def calculate_total_revenue(transactions):
2     total = 0.0
3     for t in transactions:
4         if t.get('status') == 'completed':
5             amt = t.get('amount')
6             if isinstance(amt, (int, float)):
7                 total += float(amt)
8             else:
9                 try:
10                     total += float(amt)
11                 except Exception:
12                     continue
13     return total
14

```

- Notes: CSV parsing improved to convert amounts safely and ignore invalid entries; average calculation protected against division by zero.

4.2 Project 2: TinyRAG Search System

Problem: The RAG system had poor search quality and incorrect vector math.

Agent's Approach:

1. Analyzed 'retrieval/search.py' and noticed Cosine Similarity was implemented incorrectly (dot product without normalization).
2. Corrected the logic in 'embedding.py' and 'search.py'.
3. Debugged the chunking logic to handle edge cases in 'ingest/chunker.py'.

Figure 2: Screenshot of Agent passing TinyRAG tests

Status: **PASSED** (All tests passing).

Detailed Results and Evidence:

- Commands executed:

```

1 cd test_projects/tiny_rag
2 pytest -q
3

```

- Exact test output captured:

```

1 ...
2           [100%]
3 3 passed in 0.10s

```

- Representative session logs:

```

test_projects/tiny_rag/.coding_agent_logs/session_63979a72-57a1-4d60-a211-e63
test_projects/tiny_rag/.coding_agent_logs/session_2133caff-3b56-4a9b-ac63-a98

```

- Key code fix (excerpt from `retrieval/search.py`):

```

1 def calculate_cosine_similarity(vec_a, vec_b):
2     dot_product = sum(a * b for a, b in zip(vec_a, vec_b))
3     norm_a = sum(a * a for a in vec_a) ** 0.5
4     norm_b = sum(b * b for b in vec_b) ** 0.5
5     if norm_a == 0 or norm_b == 0:
6         return 0.0
7     return dot_product / (norm_a * norm_b)
8

```

- Other changes: `embedding.py` now applies TF-IDF-like weighting, normalizes vectors and removes stopwords; `chunker.py` performs word-based chunking.

4.3 Project 3: Bigram Language Model

Problem: The model suffered from zero-probability issues (unseen words) and repetitive generation.

Agent's Approach:

1. Implemented **Laplace Smoothing** (Add-one smoothing) in 'model/bigram.py' to fix zero division errors.
2. Fixed the probability normalization logic to prevent underflow in long sequences.
3. Improved the 'sampler.py' to increase diversity in generation.

Figure 3: Screenshot of Agent passing Bigram tests

Status: **PASSED** (All tests passing).

Detailed Results and Evidence:

- Commands executed:

```

1 cd test_projects/bigram_generator
2 pytest -q
3

```

- Exact test output captured:

```

1 .....
2          [100%]
3 6 passed in 0.06s

```

- Representative session log:

`test_projects/bigram_generator/.coding_agent_logs/session_<id>.jsonl`

- Key code fixes (excerpt from `model/bigram.py`):

```

1 def get_probability(self, w1, w2):
2     V = max(1, len(self.vocab))
3     count_w1 = self.unigram_counts.get(w1, 0)
4     if count_w1 == 0:
5         return 1.0 / V
6     count_bigram = self.bigram_counts[w1].get(w2, 0)
7     return (count_bigram + 1) / (count_w1 + V)
8

```

- Diversity improvement: `generation/sampler.py` switched from greedy to weighted random sampling.
- New feature: `top_next(w, n)` added and unit-tested to return top-n next-word candidates.

5 Execution Logs

Per the submission requirements, the full conversation logs for each session are attached in the submission zip file. Below is a sample snippet of the log file structure (JSON) generated during the evaluation.

```

1 \{
2   "timestamp": "2026-01-05T11:23:10.966508Z",
3   "thread_id": "63979a72-57a1-4d60-a211-e634ce9e1d29",
4   "user_message": "Run the project's tests and fix any failing tests",
5   "agent_response": "All tests have passed successfully. There are no
6     failing tests to fix. If you need any further assistance or
7     modifications, feel free to ask!",
8   "usage": {
9     "total_tokens": 636,
10    "cost_est": 0.00011475
11  },
12   "tool_calls": []
13 \}

```

6 Video Demonstration

A 5-8 minute video demonstrating the agent's architecture, tool usage, and successful debugging of the three evaluation projects has been recorded.

Video Link: [INSERT LINK HERE OR STATE "INCLUDED IN ZIP"]

7 Detailed Test Results

Below are the exact test run summaries produced during evaluation (captured from the agent runs).

```

1 # bigram_generator (run inside test_projects/bigram_generator)
2 .....
3 [100%]
4 6 passed in 0.06s
5
6 # legacy_csv_processor (run inside test_projects/legacy_csv_processor)

```

```

6  ....
   [100%]
7  4 passed in 0.08s
8
9  # tiny_rag (run inside test_projects/tiny_rag)
10 ...
   [100%]
11 3 passed in 0.10s

```

8 Files Modified / Created

During debugging and tests the following project files were updated to fix issues and add features:

- `src/codingagent/agent/tools.py` — *implemented file/shell tools with safe-path checks*
- `src/codingagent/agent/backend.py` — *graph builder + TokenUsageTracker*
- `src/codingagent/agent/session.py` — *HITL flow, pruning, save/load, logging, whitelist auto-approve*
- `src/codingagent/cli/interactive.py` — *CLI help and commands (/save, /load)*
- `test_projects/bigram_generator/model/bigram.py` -- Laplace smoothing, log-prob scoring, top_next method
- `test_projects/bigram_generator/generation/sampler.py` -- weighted sampling for diversity
- `test_projects/bigram_generator/tests/test_bigram_top_next.py` -- new unit test
- `test_projects/legacy_csv_processor/src/file_handler.py` -- robust CSV parsing and amount conversion
- `test_projects/legacy_csv_processor/src/analyzer.py` -- handle numeric conversion and empty files
- `test_projects/tiny_rag/ingest/chunker.py` -- chunking by words with overlap
- `test_projects/tiny_rag/retrieval/embedding.py` -- TF-IDF style weighting, normalization, stopword removal
- `test_projects/tiny_rag/retrieval/search.py` -- correct cosine similarity computation

9 Usage Report

The agent reports token usage at each response. Example aggregate usage for the runs above (representative single-session numbers):

- `tiny_rag` test run: 636 tokens (estimated cost \$0.00011)
- additional interactive debugging runs: typical turns 600–2500 tokens depending on steps

10 How to Reproduce

Follow these steps on a machine with Python 3.11+:

1. Create venv and activate:

```
1 python3.11 -m venv .venv
2 source .venv/bin/activate
3
```

2. Install package:

```
1 pip install -e .
2
```

3. Provide OpenAI credentials (do NOT commit):

```
1 printf 'OPENAI_API_KEY=sk-...\nOPENAI_API_BASE=https://api.
   openai.com/v1\n' > .env
2
```

4. Run agent for a project:

```
1 coding-agent chat -p ./test_projects/tiny_rag
2
```

5. Use `/save` and `/load` to persist/restore session state.

11 Deliverables

The final submission bundle will include:

- Full source code (the `src/` folder)
- Corrected evaluation projects (`test_projects/`)
- All session logs (`.coding_agent_logs/` inside each test project)
- This report (PDF compiled from `report.tex`)
- Short usage report and the recorded demonstration video

12 Comprehensive Project Description (Appendix)

This appendix contains a complete, ordered description of the Coding Agent project, mirroring the assignment requirements and the implementation details used in this submission.

12.1 Project Overview

The goal of this project is to build an autonomous **Coding Agent** that has **Agency** — i.e., the ability to explore a codebase, edit code, execute tests and shell commands, and iterate until the requested task is completed.

Primary Capabilities

1. **Explore:** Inspect and list project files and folders to understand structure.
2. **Edit:** Create and overwrite files to implement fixes or new features.
3. **Execute:** Run shell commands such as ‘pytest’ to verify changes.
4. **Iterate:** Read test failures, update code, and re-run tests until green.

12.2 System Architecture

- **Core LLM:** The agent uses an LLM (configured via ‘`langchain_openai.ChatOpenAI`’) as the reasoning engine.
- **Orchestration:** LangGraph implements a stateful ReAct-style graph that interleaves “thinking” (LLM) and “acting” (tools).
- **CLI:** A Rich-based CLI provides a user interface for natural-language commands and HITL confirmations.
- **Persistence:** ‘MemorySaver’ checkpointer stores conversation and state during a session.

12.3 Key Components and Capabilities

12.3.1 A. Toolset (The “Hands”)

The concrete tools available to the agent are implemented in ‘`src/coding_agent/agent/tools.py`’ and include

‘`read_file(path)`’: returns file contents (guards against paths outside project root).
‘`list_files(dir)`’: lists files and directories (skip hidden and ignored folders).
‘`search_files(pattern)`’: recursive glob search for target files.
‘`create_file(path, content)`’: creates a new file (HITL – controlled).
‘`overwrite_file(path, content)`’: overwrites an existing file (HITL – controlled).
‘`execute_shell(command)`’: runs commands in a timeout-limited subprocess (HITL – controlled).

12.3.2 B. Graph Design and State Management (The “Brain”)

- The graph is created via `create_react_agent(...)` in `backend.py` and is configured to interrupt the `tools`.
- A `TokenUsageTracker` callback collects token usage from LLM responses and estimates cost.
- `MemorySaver` is used (when available) for persisting state across turns; each session uses a unique `thread_id`.

12.3.3 C. Human-In-The-Loop (Safety)

- Sensitive tools are defined in `SENSITIVE_TOOLS = "create_file", "overwrite_file", "execute"`.
- When the graph requests a sensitive tool, execution is paused and the CLI asks the user for confirmation showing the tool name and arguments.
- Two safety helpers are provided:
 - `CODING_AGENT_AUTO_APPROVE = 1` to auto-approve all sensitive tools (use caution).
 - `CODING_AGENT_AUTO_APPROVE_WHITELIST = tool1, tool2` to auto-approve only listed tools.
- If denied, a `ToolMessage` with an error is injected into the state so the agent can react.

12.4 Memory Management

- Conversation history lives in the graph state and `MemorySaver`.
- Pruning: large `ToolMessage` contents (over 2000 chars) are truncated (head + tail) after each turn to keep context manageable.
- Save/Load: `/save` attempts to pickle the checkpoint storage; if that fails, a JSON summary (messages + usage) is written as a fallback. `/load` tries to restore pickle first and falls back to loading the JSON summary (partial restore).

12.5 Evaluation Projects (detailed checklist)

For each evaluation project the agent:

1. Lists files and inspects source code.
2. Runs the project test suite (via `execute_shell("pytest")`).
2. Reads failing tracebacks and identifies buggy functions.

3. Writes fixes using `overwrite_file('HITLapprovalrequiredunlesswhitelisted')`.
3. Re-runs tests until they pass.

Project 1: Legacy CSV Processor

- Identified parsing errors and non-numeric ‘amount’ handling in `file_handler.py`.
- Fixed numeric conversion and empty-file handling in `analyzer.py`.
- Tests: ‘4 passed’ (see Detailed Test Results section).

Project 2: TinyRAG Search System

- Fixed cosine similarity normalization, applied TF-IDF-like weighting, removed stopwords, and corrected chunking.
- Tests: ‘3 passed’.

Project 3: Bigram Language Model

- Implemented Laplace smoothing, log-prob scoring, and sampled generation for diversity.
- Added `top_n_ext_convenience_method_and_test`.
- Tests: ‘6 passed’.

12.6 Bonus Features

- **Usage Tracker:** Implemented and appended usage summary to responses.
- **Smart Context:** Pruning implemented post-turn for large file contents.
- **Session Save/Load:** Implemented with pickle + JSON fallback; works for practical resume and audit.

12.7 Reproducibility and Deliverables

- Reproduction steps are included in the report (see “How to Reproduce”).
- Deliverables to produce: source, corrected test projects, logs, report PDF, demo video.

13 Methods and Libraries (Implementation Details)

This section summarizes the methodological choices and libraries used in the implementation of the Coding Agent. It maps directly to the assignment requirements and documents why each choice was made and how it was used.

13.1 Graph-Based Control Flow

- **Why:** Agents require the ability to loop (plan, act, observe, revise). A graph provides explicit nodes and edges for this cyclic logic.
- **How:** LangGraph’s ReAct-style agent is instantiated in ‘backend.py’ via `create_react_agent(...)` with `interrupt_before = ["tools"]` so the graph can pause before tool execution.
- **State:** The graph state (messages, tool calls, partial plan) is persisted in a checkpoint (MemorySaver when available) so subsequent turns can reference earlier context.

13.2 Human-in-the-Loop (HITL)

- **Sensitive Tools:** `SENSITIVE_TOOLS = ["create_file", "overwrite_file", "execute_shell"]`; the user must approve these.
- **Approval Flow:** The ‘InteractiveSession’ inspects pending tool calls, prints a panel describing the tool and its arguments, and asks the user for approval with ‘Prompt.ask’.
- **Auto-approval Options:**
 - `CODING_AGENT_AUTO_APPROVE = 1` — `global auto_approve(use_with_caution)`.
 - `CODING_AGENT_AUTO_APPROVE_WHITELIST = ["execute_shell", "create_file"]` — `whitelist specific tools for safe auto-approval`.
- **Denied Actions:** If the user denies a tool request, the session injects a ‘ToolMessage’ error into the graph state so the agent can re-plan.

13.3 Libraries & Frameworks

13.3.1 LangGraph

- **Role:** Orchestration of the multi-step, stateful agent logic; supports looping and interrupt points.
- **Usage:** `create_react_agent(...)` is used to wire tools and the LLM into a ReAct agent; we rely on the

13.3.2 LangChain

- **Role:** Tool abstraction and LLM client helpers; '@tool' decorator (or a fall-back) is used in 'tools.py'.
- **Usage:** Tools such as 'read_file', 'create_file' are implemented as functions that the graph can call.

13.3.3 Rich

- **Role:** Nicely formatted CLI: 'Console', 'Panel', and 'Prompt' are used to display agent messages, help, and to request confirmations.
- **Usage:** 'interactive.py' uses Rich to present the welcome screen, help text, and to show HITL prompts.

13.3.4 LLM (OpenAI / Chat Client)

- **Model:** Configured via 'langchain_openai.ChatOpenAI' (we used 'gpt - 4o - mini' in examples; 'gpt - 5 - mini' was the suggested model).
- **Role:** Performs reasoning, creating plans, reading error traces, and proposing code changes. The LLM is not fine-tuned — it is guided by system prompts and the graph state.
- **Usage Tracker:** 'TokenUsageTracker' monitors LLM outputs to estimate token usage and cost per turn.

13.3.5 Other Utility Libraries

- 'python-dotenv' for loading '.env' keys.
- 'pytest' for evaluation and test automation.
- 'pathlib'/'subprocess' for safe file and shell operations in tools.

13.4 Python Environment

- **Recommended:** Python 3.11+.
- **Install:** 'pip install -e .' will install required packages from 'pyproject.toml'.

13.5 Why these choices?

- Graph-based control provides explicit interrupt points and memory management, which linear chains cannot offer reliably.
- HITL ensures safety when executing destructive operations; whitelist and auto-approve options enable safe automation where desired.

- Rich gives a professional CLI which is essential for human supervisors to make quick decisions.
- Using standard LLM APIs keeps the agent flexible and model-agnostic.